

Routing and server rendering

Module 4 - Lesson 1

Overview

Next.js routes are files. The App Router introduces layouts, nested routes, and server components so that most of your page can render on the server and ship as HTML.

Key Concepts

- Files in the `app/` directory become routes; folder names become URL segments.
- `layout.tsx` wraps child routes and persists across navigation, keeping shared UI efficient.
- `page.tsx` renders the specific route; `loading.tsx` and `error.tsx` add states per route.
- Dynamic routes use `[param]` folders, read via the `params` prop.
- Server Components render on the server by default: they can read databases directly and send less JavaScript.
- Client Components add 'use client' for interactivity such as `onClick` and `useState`.

Example

```
app/  
  layout.tsx      # shared shell  
  dashboard/  
    layout.tsx    # dashboard shell  
    page.tsx      # /dashboard  
    [id]/page.tsx # /dashboard/:id
```

Summary

File-based routing makes every page predictable. The App Router composes layouts and pages, and Server Components shift rendering to the server so clients download less code.

Practice Checklist

- Create a route with a nested layout and two pages.
- Add a dynamic `[id]` route and read the `params` prop.
- Move one data-fetching component to a Server Component and confirm it renders server-side.